AETHER AI

COMPLETE PROJECT DOCUMENTATION

Source-grounded technical reference

For B.Tech Computer Science & Engineering training

Repository audit: 13 November 2026

Implementation status is distinguished from plans and placeholders.

No credentials or local database contents are reproduced.

TECHNICAL DOCUMENTATION - NOT THE FINAL TRAINING REPORT

Contents

1. Project introduction . . . 3
2. Background and objectives . . . 3
3. Problem statement and proposed solution . . . 3
4. Scope and implementation-status method . . . 3
5. Complete project directory structure . . . 5
6. Technology stack . . . 9
7. Overall system architecture . . . 9
8. Frontend architecture and UI components . . . 10
9. Backend architecture and modules . . . 11
10. Desktop application architecture . . . 12
11. AI agent and intelligence architecture . . . 12
12. Voice assistant and speech processing . . . 13
13. Text command processing . . . 14
14. Natural-language understanding and intent recognition . . . 15
15. Floating browser architecture and implementation . . . 16
16. YouTube and video automation . . . 17
17. Other browser and desktop automation . . . 18
18. AI model integrations and inference mechanisms . . . 19
19. Database, storage and memory management . . . 19
20. API documentation and integrations . . . 20
21. Security and privacy mechanisms . . . 22
22. Feature-by-feature implementation details . . . 22
23. Important classes, functions and source references . . . 23
24. Internal workflows and architecture diagrams . . . 24
25. Testing methodology and actual test cases . . . 25
26. Setup, configuration and operating model . . . 26
27. Technical challenges and engineering solutions . . . 26

28. Results, current status and known limitations 27

29. Future enhancements and scope 27

30. Technical glossary 28

Appendix A. Source verification notes 29

Appendix B. API route source index 29

Appendix C. Test inventory and limits 29

Appendix D. Reading the diagrams 30

1. Project introduction

Aether AI is a local-first desktop-assistant project organized around a React and TypeScript user interface, a FastAPI Python service, and an optional Tauri desktop shell. The implemented product combines a streaming chat endpoint, local speech transcription, search-backed retrieval, a browser surface, and local history or vector-store features. The repository is a mixed-stage engineering project: several application flows are concrete, while agent autonomy, tool safety, plugin execution, and some screens remain interfaces or placeholders.

This document describes what can be verified in the repository snapshot, not what a production deployment might eventually provide. It identifies implementation evidence, runtime dependencies, tests present, a small frontend test run performed for this documentation, and known gaps. It is intended as a technical reference for a later training-report author; it is not the final industrial-training report.

Source references: README.md; backend/README.md; backend/app/main.py; frontend/src/App.tsx

2. Background and objectives

Desktop assistants often place chat, automation, speech, and browsing in separate tools. Aether's design goal is to bring these interactions into one desktop-style interface and to keep the language model and speech-recognition path local when configured with local services. Its design also explores an embedded browser that can be positioned as a floating panel and controlled by explicit text or voice commands.

Repository-level objectives visible in the implementation are: provide a React shell with chat and feature panels; expose a versioned asynchronous API; stream generated responses; keep conversation and selected metadata locally; search through a configurable SearXNG instance; transcribe audio through Faster-Whisper; and provide a Tauri bridge for native browser-window actions. These are engineering goals, not evidence that every subsystem is production-ready.

The local-first objective is bounded. Search requires network access to configured engines; model weights may be downloaded on first use; web pages are external; and frontend/backend communication still crosses HTTP on the local machine. Local inference does not by itself provide authentication, encryption at rest, or permission isolation.

Source references: Start.md; README.md; backend/app/core/settings.py; backend/app/voice/service.py

3. Problem statement and proposed solution

The problem addressed is fragmented interaction: a user may need one interface for a question, another for speech, another for search, and a separate browser for web tasks. In addition, an assistant that performs actions needs a traceable route from user input to parsing, policy checks, execution, and visible results.

Aether proposes a layered solution. React owns the visible interface and local UI state. FastAPI provides typed request/response boundaries and streaming. The chat orchestrator routes deterministic intents to specialized services, then calls the configured Ollama endpoint for generative answers. SearXNG supplies search results; SQLite and Chroma provide distinct forms of local persistence. Tauri adds desktop-specific browser-window commands.

The implemented solution is not a general operating-system agent. Tool execution is narrow, plugin execution and permission enforcement are incomplete, and the Python browser automation path has a separate role from Tauri's native embedded webview. A report should therefore describe Aether as an evolving desktop-assistant foundation with selected working pipelines rather than as an unrestricted autonomous agent.

Source references: backend/app/chat/orchestrator.py; backend/app/chat/routing.py; backend/app/tools/manager.py; frontend/src-tauri/src/main.rs

4. Scope and implementation-status method

Status labels in this document are evidence labels. Fully implemented means a concrete path is wired into the application and has direct test coverage or an observable API/UI contract; it does not mean every deployment or edge case is validated. Partially implemented means meaningful code exists but integration, safety, coverage, or behavior is limited. Planned/interface means only a scaffold, abstraction, label, or no-op exists. Unsupported means the code explicitly rejects a capability or the repository contains no implemented path for it.

A feature may be implemented in code while its dependent service is not running. For example, SearXNG requests and Ollama streaming are implemented, but a successful live search or model response depends on a local service and network configuration. No accuracy, latency, throughput, or user-study statistics are available in the repository review.

Source references: Status derived from source and test inventory; not from marketing labels.

Table 4.1. Source-based implementation status

Capability	Status	Evidence and boundary
FastAPI app, versioned router, health/system surfaces	Implemented	Lifecycle, routes, schemas, middleware and system snapshot are wired.
Chat with Ollama streaming	Implemented, service-dependent	SSE and configured Ollama NDJSON path; stream flag is not an output-mode switch.
Deterministic intent/routing and memory recall	Implemented, heuristic	Keyword/rule classification and selected Chroma recall; not a trained NLU model.
Voice transcription	Implemented, service-dependent	Browser MediaRecorder uploads audio to a local FastAPI Faster-Whisper service.
Floating browser in Tauri desktop	Implemented, platform-specific	Child WebView2 plus Tauri commands; native top-most behavior is not established by state alone.
Backend Playwright browser control	Partial	Attaches to CDP; assumes the embedded browser is available at the configured local port.
Video discovery and save/list	Implemented, service-dependent	SearXNG video category; metadata stored in Chroma.
YouTube DOM automation	Partial	Search/navigation/playback/info code exists; dependent on live site DOM and narrow supported actions.
Image search and history	Implemented, service-dependent	SearXNG retrieval plus Chroma history; download proxy has destination-safety gaps.
Screenshot listing and CRUD	Implemented	Captured data is stored in SQLite; UI wording suggesting memory-only storage is inaccurate.
General agent, workflow, plugin runtime	Planned/interface	Modules are placeholders or descriptors, not general execution frameworks.
Authorization and encryption	Not implemented	Permission/encryption classes are placeholders; no API authentication is shown.
Dashboard/module pages	Placeholder	Dashboard renders null; module page is a label, and layout does not render its Outlet.

5. Complete project directory structure

The listing below is derived from tracked repository paths at generation time. It includes source, test, build-configuration, and documentation paths. It intentionally excludes local databases and their records, ignored virtual environments and dependency trees, build caches, environment files, and generated binary build output. Those exclusions preserve privacy and keep the tree useful as an engineering map.

There is no tracked root docs/ directory in this snapshot, although the root README describes one. Some README directory descriptions also refer to folders that are not present in the inspected source tree. The tracked structure, rather than a stale prose inventory, is the authoritative directory map.

Source references: README.md; backend/README.md; tracked file list.

```
AetherAI/
|-- backend/
|   |-- app/
|   |   |-- agents/
|   |   |   |-- __init__.py
|   |   |   `-- planner.py
|   |   |-- api/
|   |   |   |-- v1/
|   |   |   |   |-- routes/
|   |   |   |   |   |-- __init__.py
|   |   |   |   |   |-- browser.py
|   |   |   |   |   |-- chat.py
|   |   |   |   |   |-- health.py
|   |   |   |   |   |-- images.py
|   |   |   |   |   |-- memory.py
|   |   |   |   |   |-- monitoring.py
|   |   |   |   |   |-- screenshots.py
|   |   |   |   |   |-- system.py
|   |   |   |   |   `-- videos.py
|   |   |   |   |-- __init__.py
|   |   |   |   `-- router.py
|   |   |   |-- __init__.py
|   |   |   `-- dependencies.py
|   |   |-- browser/
|   |   |   |-- __init__.py
|   |   |   |-- browser_automation.py
|   |   |   |-- browser_config.py
|   |   |   |-- browser_controller.py
|   |   |   |-- browser_events.py
|   |   |   |-- browser_interaction.py
|   |   |   |-- browser_manager.py
|   |   |   |-- browser_navigation.py
|   |   |   |-- browser_registry.py
|   |   |   |-- browser_session.py
|   |   |   |-- browser_state.py
|   |   |   |-- context.py
|   |   |   |-- download.py
|   |   |   |-- exceptions.py
|   |   |   |-- floating_manager.py
|   |   |   |-- manager.py
|   |   |   |-- models.py
|   |   |   |-- navigation.py
|   |   |   |-- schemas.py
|   |   |   |-- screenshot.py
|   |   |   |-- screenshot_store.py
|   |   |   |-- service.py
|   |   |   |-- state.py
|   |   |   |-- storage.py
|   |   |   |-- tab.py
|   |   |   `-- window_manager.py
|   |   |-- chat/
|   |   |   |-- __init__.py
|   |   |   |-- confidence.py
|   |   |   |-- intent.py
|   |   |   |-- memory_manager.py
|   |   |   |-- monitoring_logger.py
|   |   |   |-- orchestrator.py
|   |   |   |-- prompt_builder.py
|   |   |   |-- response_generator.py
|   |   |   |-- routing.py
|   |   |   |-- schemas.py
|   |   |   `-- service.py
|   |   |-- core/
|   |   |   |-- container.py
|   |   |   |-- interfaces.py
|   |   |   |-- logging.py
|   |   |   `-- settings.py
|   |   |-- database/
|   |   |   |-- __init__.py
|   |   |   |-- base.py
|   |   |   |-- chroma.py
|   |   |   |-- redis.py
|   |   |   `-- session.py
|   |   |-- images/
|   |   |   |-- __init__.py
|   |   |   |-- history_store.py
|   |   |   |-- schemas.py
```

```
|   |   |   `-- service.py
|   |   |-- llm/
|   |   |   |-- adapters/
|   |   |   |   `-- README.md
|   |   |   |-- __init__.py
|   |   |   |-- ollama_provider.py
|   |   |   `-- provider_manager.py
|   |   |-- memory/
|   |   |   |-- __init__.py
|   |   |   |-- chroma_manager.py
|   |   |   |-- history_store.py
|   |   |   `-- manager.py
|   |   |-- middleware/
|   |   |   |-- __init__.py
|   |   |   |-- exception_handler.py
|   |   |   `-- request_id.py
|   |   |-- models/
|   |   |   |-- __init__.py
|   |   |   |-- audit_event.py
|   |   |   `-- conversation.py
|   |   |-- monitoring/
|   |   |   |-- __init__.py
|   |   |   `-- service.py
|   |   |-- plugins/
|   |   |   |-- __init__.py
|   |   |   `-- manager.py
|   |   |-- rag/
|   |   |   |-- __init__.py
|   |   |   `-- engine.py
|   |   |-- repositories/
|   |   |   |-- __init__.py
|   |   |   `-- base.py
|   |   |-- schemas/
|   |   |   |-- __init__.py
|   |   |   |-- errors.py
|   |   |   |-- health.py
|   |   |   `-- system.py
|   |   |-- security/
|   |   |   |-- __init__.py
|   |   |   |-- audit_logger.py
|   |   |   |-- encryption_service.py
|   |   |   |-- permission_manager.py
|   |   |   `-- secret_manager.py
|   |   |-- services/
|   |   |   |-- __init__.py
|   |   |   `-- system_service.py
|   |   |-- tools/
|   |   |   |-- __init__.py
|   |   |   `-- manager.py
|   |   |-- videos/
|   |   |   |-- __init__.py
|   |   |   |-- command_router.py
|   |   |   |-- schemas.py
|   |   |   |-- service.py
|   |   |   `-- storage.py
|   |   |-- voice/
|   |   |   |-- commands.py
|   |   |   |-- routes.py
|   |   |   |-- schemas.py
|   |   |   `-- service.py
|   |   |-- web/
|   |   |   |-- __init__.py
|   |   |   `-- search.py
|   |   |-- workflow/
|   |   |   |-- __init__.py
|   |   |   `-- engine.py
|   |   |-- youtube/
|   |   |   |-- __init__.py
|   |   |   |-- models.py
|   |   |   |-- README.md
|   |   |   |-- youtube_command_router.py
|   |   |   `-- youtube_manager.py
|   |   |-- __init__.py
|   |   `-- main.py
|   |-- tests/
|   |   |-- README.md
|   |   |-- test_browser.py
|   |   |-- test_browser_active_page.py
|   |   |-- test_browser_window_system.py
|   |   |-- test_centralized_browser.py
|   |   |-- test_chat.py
|   |   |-- test_health.py
|   |   |-- test_memory.py
|   |   |-- test_monitoring.py
|   |   |-- test_orchestration_routing.py
|   |   |-- test_persistence.py
|   |   |-- test_screenshot_store.py
|   |   |-- test_videos.py
|   |   |-- test_voice.py
|   |   |-- test_web_search.py
|   |   `-- test_youtube.py
|   |-- .editorconfig
|   |-- alembic.ini
|   |-- pyproject.toml
|   |-- pytest.ini
|   `-- README.md
```

```
|-- docker/
|   |-- searxng/
|   |   |-- searxng/
|   |   |   `-- settings.yml
|   |   `-- docker-compose.yml
|   `-- README.md
|-- frontend/
|   |-- public/
|   |   `-- assets/
|   |       |-- aether-logo.png
|   |       |-- DisplayVideo.mp4
|   |       |-- logo.png
|   |       |-- StartingVideo.mp4
|   |       `-- StartingVideo2.mp4
|   |-- src/
|   |   |-- components/
|   |   |   |-- aether/
|   |   |   |   |-- AetherBackground.tsx
|   |   |   |   |-- AetherLoader.tsx
|   |   |   |   |-- AetherLogo.tsx
|   |   |   |   |-- AetherStatus.tsx
|   |   |   |   |-- index.ts
|   |   |   |   `-- TopNavAether.test.tsx
|   |   |   |-- avatar/
|   |   |   |   |-- AnimationEngine.ts
|   |   |   |   |-- Avatar.tsx
|   |   |   |   |-- AvatarAssetManager.test.ts
|   |   |   |   |-- AvatarAssetManager.ts
|   |   |   |   |-- AvatarAudio.ts
|   |   |   |   |-- AvatarController.ts
|   |   |   |   |-- AvatarHooks.ts
|   |   |   |   |-- AvatarLipSync.test.ts
|   |   |   |   |-- AvatarLipSync.ts
|   |   |   |   |-- AvatarState.ts
|   |   |   |   |-- AvatarStyles.css
|   |   |   |   |-- EmotionEngine.test.ts
|   |   |   |   |-- EmotionEngine.ts
|   |   |   |   |-- ExpressionResolver.ts
|   |   |   |   `-- index.ts
|   |   |   |-- bottom/
|   |   |   |   `-- BottomNavbar.tsx
|   |   |   |-- browser/
|   |   |   |   |-- browser.css
|   |   |   |   |-- BrowserLauncher.tsx
|   |   |   |   |-- BrowserProvider.tsx
|   |   |   |   |-- BrowserWindow.tsx
|   |   |   |   |-- FloatingToolbar.tsx
|   |   |   |   |-- index.ts
|   |   |   |   `-- MiniPlayer.tsx
|   |   |   |-- chat/
|   |   |   |   |-- Markdown/
|   |   |   |   |   |-- blocks/
|   |   |   |   |   |   |-- AnimatedBlock.tsx
|   |   |   |   |   |   |-- CalloutBlock.tsx
|   |   |   |   |   |   |-- CodeBlock.tsx
|   |   |   |   |   |   |-- CustomBlockQuote.tsx
|   |   |   |   |   |   |-- CustomImage.tsx
|   |   |   |   |   |   |-- CustomLink.tsx
|   |   |   |   |   |   |-- CustomTable.tsx
|   |   |   |   |   |   `-- Heading.tsx
|   |   |   |   |   |-- CustomLink.test.tsx
|   |   |   |   |   |-- helpers.ts
|   |   |   |   |   |-- MarkdownAnimations.ts
|   |   |   |   |   |-- MarkdownMessage.tsx
|   |   |   |   |   |-- types.ts
|   |   |   |   |   `-- utils.ts
|   |   |   |   |-- ChatPanelAether.tsx
|   |   |   |   |-- LightningLoader.tsx
|   |   |   |   |-- MarkdownMessage.test.tsx
|   |   |   |   `-- VoiceIntegration.test.tsx
|   |   |   |-- files/
|   |   |   |   |-- FilesModal.tsx
|   |   |   |   |-- FilesProvider.tsx
|   |   |   |   |-- index.ts
|   |   |   |   `-- types.ts
|   |   |   |-- image-search/
|   |   |   |   |-- ImageSearchModal.tsx
|   |   |   |   |-- ImageSearchProvider.tsx
|   |   |   |   |-- index.ts
|   |   |   |   `-- types.ts
|   |   |   |-- memory/
|   |   |   |   `-- MemoryModal.tsx
|   |   |   |-- monitor/
|   |   |   |   `-- MonitorPanel.tsx
|   |   |   |-- plugins/
|   |   |   |   `-- PluginShowcaseModal.tsx
|   |   |   |-- status/
|   |   |   |   `-- StatusPanel.tsx
|   |   |   |-- ui/
|   |   |   |   |-- button.tsx
|   |   |   |   `-- card.tsx
|   |   |   |-- videos/
|   |   |   |   |-- index.ts
|   |   |   |   |-- types.ts
|   |   |   |   |-- VideoSearchContext.ts
|   |   |   |   |-- VideoSearchProvider.tsx
```

```
|   |   |   |   `-- VideosModal.tsx
|   |   |   `-- BackgroundField.tsx
|   |   |-- hooks/
|   |   |   |-- index.ts
|   |   |   |-- useBrowser.ts
|   |   |   |-- useChat.ts
|   |   |   |-- useDock.ts
|   |   |   |-- useKeyboard.ts
|   |   |   |-- useMetrics.ts
|   |   |   |-- useMonitoring.ts
|   |   |   |-- useSmartScroll.test.ts
|   |   |   |-- useSmartScroll.ts
|   |   |   |-- useSystemSnapshot.ts
|   |   |   |-- useTheme.ts
|   |   |   |-- useVoiceAssistant.ts
|   |   |   |-- useWindowSize.ts
|   |   |   `-- useWorkspace.ts
|   |   |-- layouts/
|   |   |   `-- DesktopLayout.tsx
|   |   |-- pages/
|   |   |   |-- Dashboard.tsx
|   |   |   `-- ModulePage.tsx
|   |   |-- services/
|   |   |   |-- apiClient.ts
|   |   |   `-- systemService.ts
|   |   |-- stores/
|   |   |   |-- browserStore.ts
|   |   |   `-- commandStore.ts
|   |   |-- styles/
|   |   |   |-- animations.css
|   |   |   |-- effects.css
|   |   |   |-- glass.css
|   |   |   |-- globals.css
|   |   |   |-- scrollbar.css
|   |   |   |-- typography.css
|   |   |   `-- variables.css
|   |   |-- test/
|   |   |   `-- setup.ts
|   |   |-- types/
|   |   |   |-- browser.ts
|   |   |   |-- chat.ts
|   |   |   |-- dashboard.ts
|   |   |   |-- index.ts
|   |   |   `-- monitoring.ts
|   |   |-- utils/
|   |   |   `-- cn.ts
|   |   |-- voice/
|   |   |   |-- browserCommands.test.ts
|   |   |   |-- browserCommands.ts
|   |   |   |-- engine.test.ts
|   |   |   |-- engine.ts
|   |   |   |-- formatter.test.ts
|   |   |   |-- formatter.ts
|   |   |   |-- index.ts
|   |   |   |-- languages.test.ts
|   |   |   |-- languages.ts
|   |   |   |-- monitoring.test.ts
|   |   |   |-- monitoring.ts
|   |   |   |-- providers.ts
|   |   |   `-- types.ts
|   |   |-- App.tsx
|   |   |-- main.tsx
|   |   |-- routes.tsx
|   |   `-- vite-env.d.ts
|   |-- src-tauri/
|   |   |-- capabilities/
|   |   |   `-- default.json
|   |   |-- gen/
|   |   |   `-- schemas/
|   |   |       |-- acl-manifests.json
|   |   |       |-- capabilities.json
|   |   |       |-- desktop-schema.json
|   |   |       `-- windows-schema.json
|   |   |-- icons/
|   |   |   `-- icon.ico
|   |   |-- src/
|   |   |   |-- browser/
|   |   |   |   |-- commands.rs
|   |   |   |   |-- mod.rs
|   |   |   |   `-- window.rs
|   |   |   `-- main.rs
|   |   |-- build.rs
|   |   |-- Cargo.lock
|   |   |-- Cargo.toml
|   |   |-- create_icon.py
|   |   `-- tauri.conf.json
|   |-- .editorconfig
|   |-- .prettierrc
|   |-- eslint.config.js
|   |-- index.html
|   |-- package-lock.json
|   |-- package.json
|   |-- postcss.config.js
|   |-- README.md
|   |-- tailwind.config.d.ts
|   |-- tailwind.config.js
```

```
|   |-- tailwind.config.ts
|   |-- tsconfig.json
|   |-- tsconfig.node.json
|   |-- vite.config.d.ts
|   |-- vite.config.js
|   |-- vite.config.ts
|   `-- vitest.config.ts
|-- .gitignore
|-- package-lock.json
|-- package.json
|-- README.md
`-- Start.md

[Excluded from this listing: ignored dependency trees, generated caches/build output,
local runtime databases and records, and environment files. Their contents were not read.]
```

6. Technology stack

The stack is multi-language and uses local services rather than one monolithic runtime. Python contains the API and service layer; TypeScript/React contains the interactive interface; Rust/Tauri contains desktop-native commands. SearXNG, Ollama, Chroma, and SQLite are separate service or persistence components. Dependency declarations establish intended support, not proof that each optional integration is active in every code path.

The backend requires Python 3.13 or newer in pyproject.toml and uses FastAPI, Pydantic v2, SQLAlchemy async, aiosqlite, ChromaDB, Playwright, Faster-Whisper, httpx, Redis client libraries, and standard test/lint tools. Although PostgreSQL, Redis, Alembic, and clean-architecture interfaces are present in configuration or dependencies, the audited active persistence paths are SQLite and Chroma; startup calls SQLAlchemy create_all rather than running an Alembic migration workflow.

The frontend package declares React 18, TypeScript, Vite, React Router, Zustand, TanStack React Query, Framer Motion, Tailwind CSS, Lucide icons, Markdown renderers, and Tauri v2 APIs. Vitest and Testing Library run unit/component tests in jsdom. The desktop crate uses Rust 2021 and Tauri v2 with serde and Tokio. The local Docker Compose file runs SearXNG only; it is not a complete compose deployment for all README-listed services.

Source references: backend/pyproject.toml; frontend/package.json; frontend/src-tauri/Cargo.toml; docker/searxng/docker-compose.yml

Table 6.1. Technology inventory and actual role

Layer	Technology	Role and verified boundary
Backend	Python 3.13+, FastAPI, Pydantic v2	Typed HTTP/WebSocket API, dependency wiring and validation.
Persistence	SQLAlchemy async + SQLite	Conversation and screenshot persistence; table creation at startup.
Vector storage	ChromaDB	Semantic memory and search-history/saved-video metadata paths.
Model runtime	Ollama	Local model generation through HTTP NDJSON; model name is configuration-dependent.
Speech recognition	Faster-Whisper	Local speech-to-text; model initialization and CPU/GPU settings are configurable.
Search	SearXNG	Web/image/video metasearch, using configured categories and JSON responses.
Browser automation	Playwright + CDP	Page automation connects to the selected embedded browser; does not launch a standalone browser in the audited path.
Frontend	React, TypeScript, Vite	Desktop-style single-page interface and component state.
Desktop	Rust, Tauri 2, WebView2	Native shell, invoke commands, attached browser webview; current implementation is Windows-centric.
Frontend state/UI	Zustand, React Query, Framer Motion, Tailwind	Browser state, system polling, animation, styling.
Testing	pytest, Vitest, Testing Library, jsdom	Backend test suite is present; frontend test suite was run once for this audit.

7. Overall system architecture

At runtime the system has several execution boundaries. The user's text or recorded audio begins in the React interface. Ordinary chat crosses HTTP to the FastAPI chat route, whose orchestration code selects a specialist branch or a language-model response. Voice recording first uses the browser/WebView microphone and sends audio to the FastAPI transcription endpoint; the resulting transcript is routed through voice commands or chat. Monitoring uses a separate WebSocket and a local in-process event bus.

The native browser surface follows a different route: React invokes Rust commands through Tauri, and Rust creates or updates a child WebView2 surface. The Python Playwright manager attaches through the configured CDP debugging port to automate a selected browser page. The frontend synchronizes active-page information to the backend; this is a coordination bridge, not a single shared browser-state object.

Persistence is likewise split. Conversation history and screenshot records use SQLAlchemy/SQLite paths. Semantic memories, image-search history, and saved-video metadata use persistent Chroma collections. The Redis client/configuration exists but is not part of the described chat persistence path. The diagram is a logical view: exact deployment topology varies with local configuration.

Source references: backend/app/main.py; backend/app/api/v1/router.py; backend/app/chat/orchestrator.py; frontend/src/App.tsx; frontend/src-tauri/src/main.rs

Aether AI logical architecture

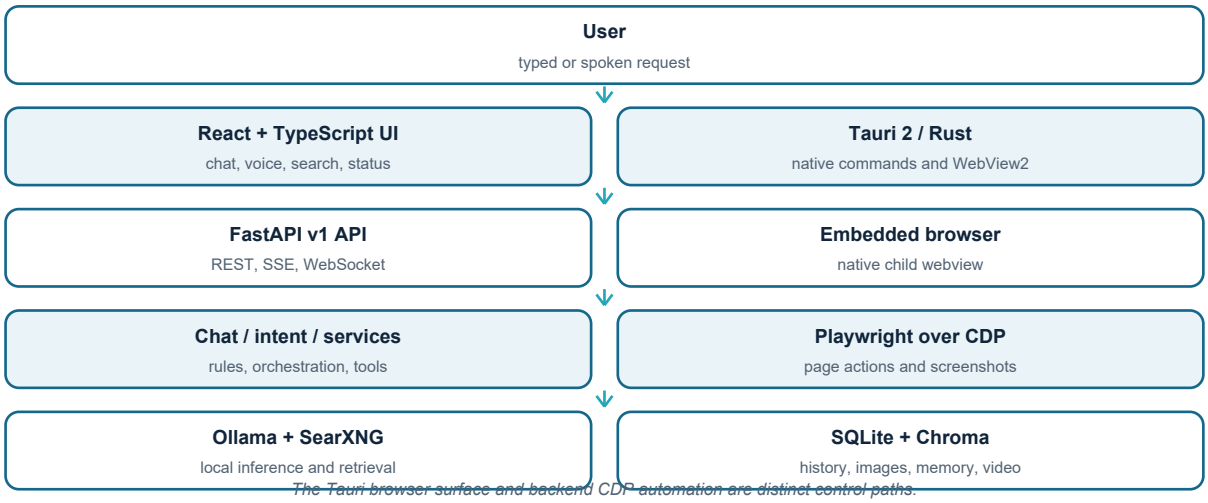


Figure 7.1. Logical components and principal dependencies.

8. Frontend architecture and UI components

The frontend bootstraps through src/main.tsx into App.tsx. App composes ImageSearchProvider, VideoSearchProvider, FilesProvider, BrowserProvider, React Query, and React Router, then mounts the global modals. The top-level route defines a DesktopLayout with a root Dashboard, a redirect from /home, and a parameterized module route.

DesktopLayout provides the visible shell: startup animation and display video, status telemetry, live monitoring, chat, the browser window/launcher, and bottom navigation. ChatPanelAether combines useChat, useVoiceAssistant, transcript/status presentation, and Markdown rendering. Browser components coordinate a draggable/resizable rectangle and native webview bounds. Image/video/files/memory components are modal-based feature surfaces.

Some visible entries are placeholders rather than working modules. Dashboard returns null. ModulePage only prints its route parameter and DesktopLayout does not render the imported Outlet, so nested route content is not composed into the shell. Tools, Tasks, Notifications, and Settings navigation buttons have no action handlers. useMetrics, useWorkspace, useDock, and useKeyboard return empty objects. PluginShowcaseModal is a technology-stack display, not a plugin manager.

The UI uses React Query to poll system snapshots every two seconds, Zustand for browser state, context providers for image/video/files state, and a WebSocket hook for events. The default API URL is a local FastAPI URL, configurable with VITE_AETHER_API_URL; monitoring optionally accepts VITE_AETHER_WS_URL. The generic API client handles JSON GET/POST/DELETE and throws on non-success HTTP responses; chat uses a separate streaming fetch implementation.

The animated avatar is integrated through AetherLogo and Avatar. Its controller maps chat/voice phases to expressions, emotion blending, animation, and mouth shapes. The rendered face is CSS-driven; although AvatarAssetManager includes asset-loading and layered-skin fallback logic, the main Avatar rendering does not display those loaded image assets. When the controller enters listening state, AvatarAudio requests its own microphone stream for amplitude analysis in addition to the voice recorder's stream.

The Files modal lists, previews, downloads, and deletes existing screenshots, but the reviewed frontend provider does not implement a capture action. Its instruction to ask for a screenshot is not itself a screenshot trigger. The backend browser command path can capture screenshots, but the frontend files pane does not call that command.

Source references: frontend/src/main.tsx; frontend/src/App.tsx; frontend/src/routes.tsx; frontend/src/layouts/DesktopLayout.tsx; frontend/src/hooks/useChat.ts; frontend/src/services/apiClient.ts

Table 8.1. User-interface surfaces and maturity

Surface	Files	Behavior / maturity
Desktop shell	layouts/DesktopLayout.tsx; components/aether/AetherBackground.tsx	Boot media, status, monitor, chat, browser, bottom navigation.
Chat	components/chat/ChatPanelAether.tsx; hooks/useChat.ts	Text submission, SSE stream, Markdown rendering, local browser command interception.
Voice	hooks/useVoiceAssistant.ts; voice/*	Capture/transcription/chat/TTS lifecycle; dependencies described in Chapter 12.
Browser window	components/browser/*; stores/browserStore.ts	Drag/resize/layout/address bar/tab controls, backed by Tauri invoke.
Video search	components/videos/*	Search, pagination, filters, saved items, open video in Aether browser.
Images and files	components/image-search/*; components/files/*	Search result viewer/history and screenshot list/download/delete.
Memory and stack	components/memory/MemoryModal.tsx; components/plugins/PluginShowcaseModal.tsx	View/search memory; stack showcase rather than executable plugin management.
Avatar	components/aether/AetherLogo.tsx; components/avatar/*	CSS avatar is rendered; controller animates state, expression and mouth. Asset pipeline is not the rendered face.
Other nav entries	components/bottom/BottomNavbar.tsx	Tools, Tasks, Notifications and Settings are inert buttons.

9. Backend architecture and modules

FastAPI application construction is in app/main.py. The lifespan configures structured logging, creates a container, initializes SQLAlchemy tables, begins Faster-Whisper loading in a background task, and shuts down voice during teardown. create_app registers middleware and exception handlers and mounts the versioned router. Dependency construction lives in core/container.py; current factories are not a fully cached dependency graph.

The API layer is separated by feature under app/api/v1/routes. Chat types and behavior live in app/chat; provider code is in app/llm; browser coordination and page automation are in app/browser; local speech processing is in app/voice; search and specialized media code are in app/web, app/images, app/videos and app/youtube. Persistence responsibilities are in app/memory, app/database, app/models, and feature-specific stores.

This organization gives useful boundaries, but directory names alone do not prove feature completeness. The general workflow engine and RAG engine are initialization stubs. The plugin manager has no executable plugins. Provider descriptors do not implement a provider-selection system. Redis helpers are not active in the primary memory/history route. System status fields for vision, internet, and tools are static labels rather than verified health checks.

Errors are handled by global exception handlers for HTTP, validation, and unexpected exceptions; a request-ID middleware adds correlation information. Route-specific validation maps some invalid requests and upstream failures to 4xx/5xx responses. Streaming chat catches internal failures and yields a user-facing event. Logging is structured to stdout and can also produce monitoring events, so transcript/personally identifying data should be reviewed before deployment.

Source references: backend/app/main.py; backend/app/core/container.py; backend/app/api/v1/router.py; backend/app/middleware/exception_handler.py; backend/app/middleware/request_id.py

Table 9.1. Backend module map

Package	Responsibility evidenced in source
api/v1/routes	Health, system, chat, browser, images, memory, monitoring, screenshots, videos.
chat	Request schemas, intent rules, routing, orchestration, prompt building, response generation.
llm	Ollama HTTP provider; provider manager is descriptive rather than multi-provider execution.
browser	Central browser manager/session, navigation/interaction, screenshot and state helpers; some legacy helpers remain.

Package	Responsibility evidenced in source
voice	Faster-Whisper service, health/transcription endpoints, deterministic local command executor.
web	SearXNG retrieval, HTML extraction, ranking/citations and caching heuristics.
memory/database/models	SQLite conversation history, Chroma semantic memory and supporting persistence config.
images/videos/youtube	Image retrieval/history/download, video retrieval/saved metadata, YouTube DOM command manager.
agents/tools/workflow/rag/plugins	Limited browser planner/tool dispatch plus placeholder or narrow subsystem foundations.
security/monitoring	In-process event bus and security interfaces; authorization/encryption/audit persistence are incomplete.

10. Desktop application architecture

The desktop shell is defined under frontend/src-tauri. Rust main.rs creates the Tauri application, installs the shell plugin, registers a BrowserWindowManager, and exposes browser commands using invoke_handler. The project is Tauri 2 with Rust 2021. Its window is configured as a resizable 1440 by 960 logical-pixel window with minimum dimensions, and the bundle is disabled in the current config.

The browser is not a separate Chrome process in the native UI path. Tauri attaches a child WebView to the main window. The React BrowserWindow is a DOM layout/toolbar which measures its content box, then calls browser_update_bounds; Rust ensures each tab webview exists at those coordinates. During drag, the child webview is hidden to avoid intercepting pointer movement. The main process sets WebView2 debugging/media arguments; this is a platform-specific integration choice with security implications described in Chapter 21.

Browser state exists in both the frontend Zustand store and Rust BrowserWindowState. Commands cover open/navigate/show/hide/minimize/maximize/close/back/forward/reload/tab switching/position/size/snap. The Rust snap command itself only restores visibility; the React component calculates the geometry. The always-on-top command updates state but does not call a native window API in the inspected Rust handler, so cross-window topmost behavior is not verified.

The Tauri capability grants core, window, webview, and shell-open permissions. The config has a null content-security policy. These are configuration facts, not a complete security boundary. The app should be reviewed with a production CSP, least-privilege Tauri capabilities, explicit webview navigation rules, and a deployment-specific permissions policy.

Source references: frontend/src-tauri/src/main.rs; frontend/src-tauri/src/browser/commands.rs; frontend/src-tauri/src/browser/window.rs; frontend/src-tauri/tauri.conf.json; frontend/src-tauri/capabilities/default.json

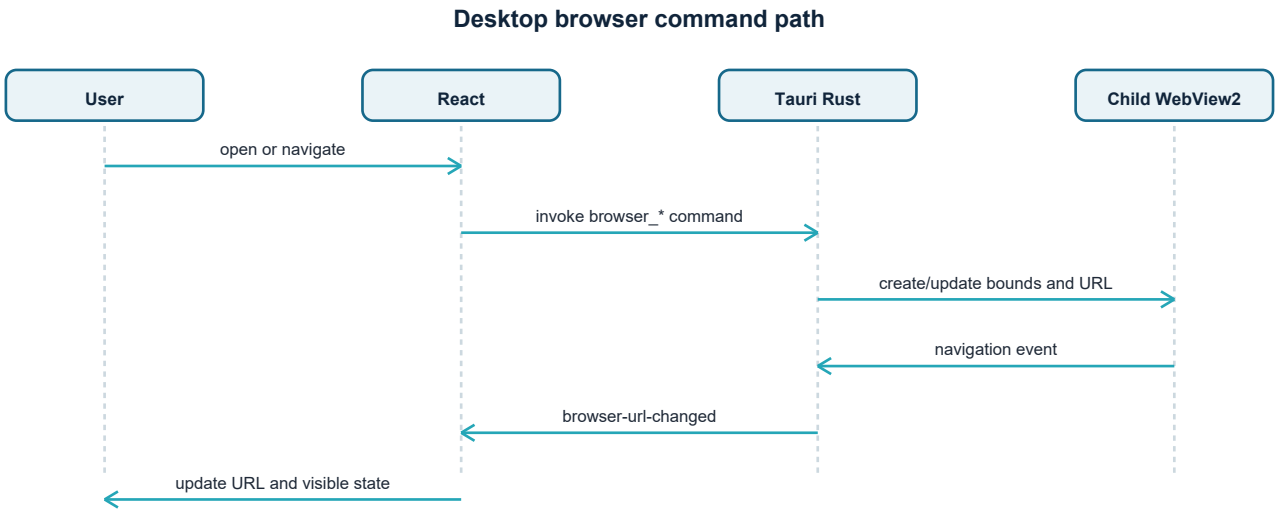


Figure 10.1. Tauri invoke and native embedded-browser control path.

11. AI agent and intelligence architecture

The functioning intelligence path is primarily an orchestrator with deterministic routing and an LLM fallback. ConversationOrchestrator processes specialized video and YouTube commands before other routes, handles deterministic greetings/identity/date cases, prepares browser plans, image searches, memory/search context, and model-backed answers. IntentClassifier uses keyword and regular-expression rules; RoutingEngine maps intent categories to service branches.

PlaceholderPlanner has real but narrow behavior: recognize browser-oriented patterns, optionally request a plan from the configured model, and fall back to a simple open/search step. PlaceholderToolManager dispatches a fixed set of browser tools. The existence of names such as agent, workflow, and RAG does not mean Aether has a general multi-step agent graph: workflow and RAG modules are stubs, plugin execution is empty, and tool authorization/consent is not implemented.

No model training pipeline, training dataset, fine-tuned Aether model, benchmark suite, or measured intent accuracy was found in the repository review. Ollama provides inference from an externally supplied local model. The intent classifier is code-defined. Chroma supplies retrieval, but the code does not establish a project-owned embedding-training process.

Current-information requests use a web-search path before generation. The evidence validator is heuristic: it checks for the presence of documents but does not rigorously establish source agreement or relevance. Search snippets may serve as fallback evidence. A responsible answer should describe this as retrieval-assisted prompting with heuristic checks, not a verified fact-checking system.

Source references: backend/app/chat/intent.py; backend/app/chat/routing.py; backend/app/chat/orchestrator.py; backend/app/agents/planner.py; backend/app/tools/manager.py; backend/app/rag/engine.py; backend/app/workflow/engine.py

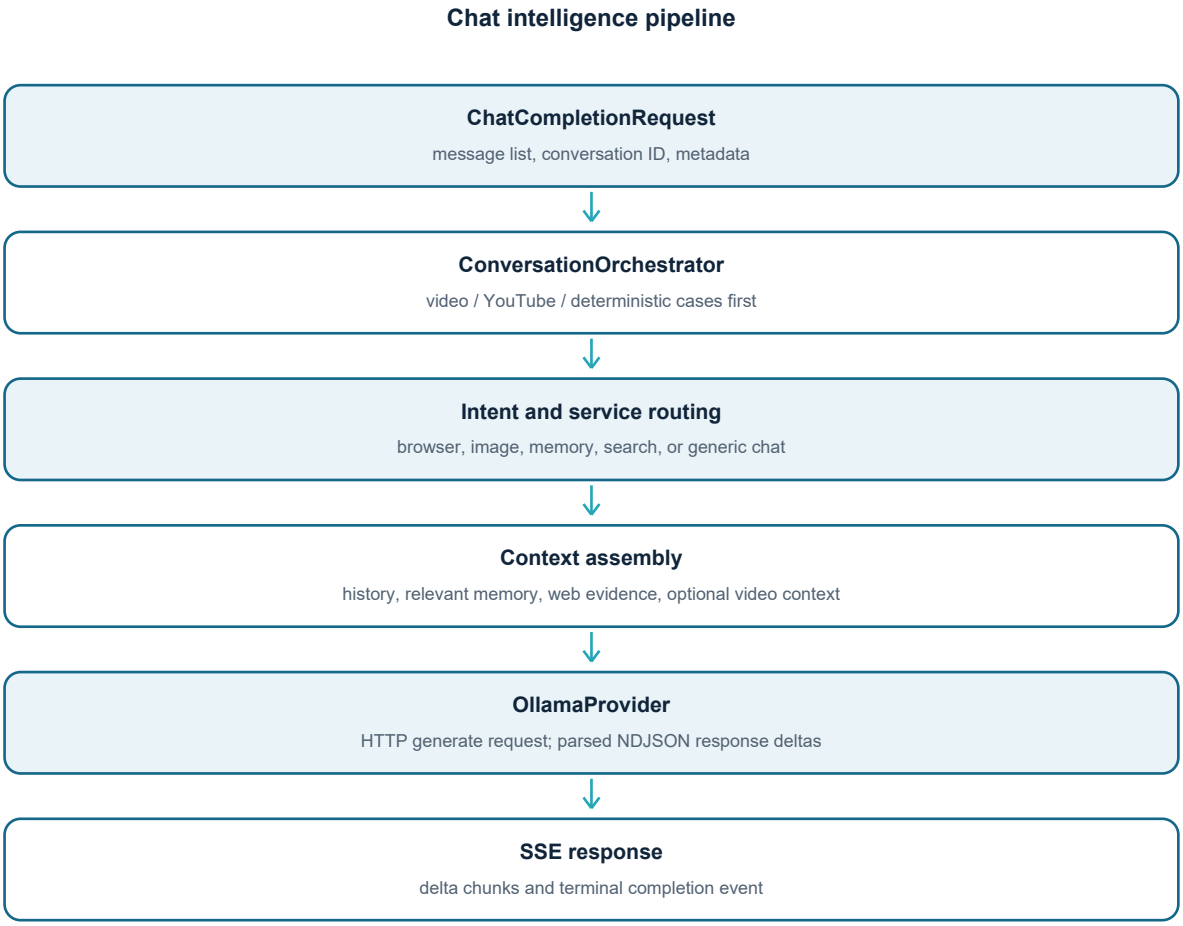


Figure 11.1. Simplified request-routing path; specialized commands are evaluated before generic model generation.

12. Voice assistant and speech processing

Voice starts in the frontend. AetherVoiceEngine maintains a state machine including starting, listening, processing_speech, understanding, executing, sending_to_chatbot, receiving_ai_response, converting_to_speech, speaking, completed, and error. The React hook subscribes to state and stops the engine on unmount. The UI presents transcripts, phase labels, errors, and a microphone toggle.

The speech-recognition provider requests a mono microphone stream with echo cancellation, noise suppression, and automatic gain control. MediaRecorder emits WebM/Opus chunks. During capture, the frontend can send a partial recording periodically for interim transcription; stopping the session sends a final multipart file to POST /api/v1/voice/transcribe. The backend loads Faster-Whisper locally, applies configured language/VAD/device/compute

settings, serializes transcription work, and returns recognized text and language. This is local HTTP transfer between frontend and backend, not browser-native speech recognition.

After transcription, a small set of stop phrases is handled locally. The frontend strips an optional Aether prefix, then calls POST /api/v1/voice/execute for deterministic local commands. If not handled, it calls the chat pipeline with source=voice and a per-session conversation ID. Chat deltas update the visible response. Browser SpeechSynthesis speaks the finished response; this browser TTS is distinct from Faster-Whisper, which is speech-to-text only.

A RegexWakeWordDetector is defined in the frontend provider module, but the engine does not use it to gate recording in the inspected source. The UI may suggest saying Aether, yet actual capture begins after the user activates voice mode and commands are processed without a demonstrated acoustic wake-word detector. Speech synthesis voices are browser/OS-provided. Language handling is a small rule set (not a general multilingual NLU model).

Backend command execution is deliberately narrow: date/time, allowlisted applications, and selected user folders. It rejects dangerous/arbitrary shell commands rather than exposing a generic command shell. There is still no API authentication around the route, so local bind and deployment policy matter.

Source references: frontend/src/voice/engine.ts; frontend/src/voice/providers.ts; frontend/src/voice/languages.ts; frontend/src/voice/formatter.ts; backend/app/voice/service.py; backend/app/voice/commands.py; backend/app/voice/routes.py

Voice workflow

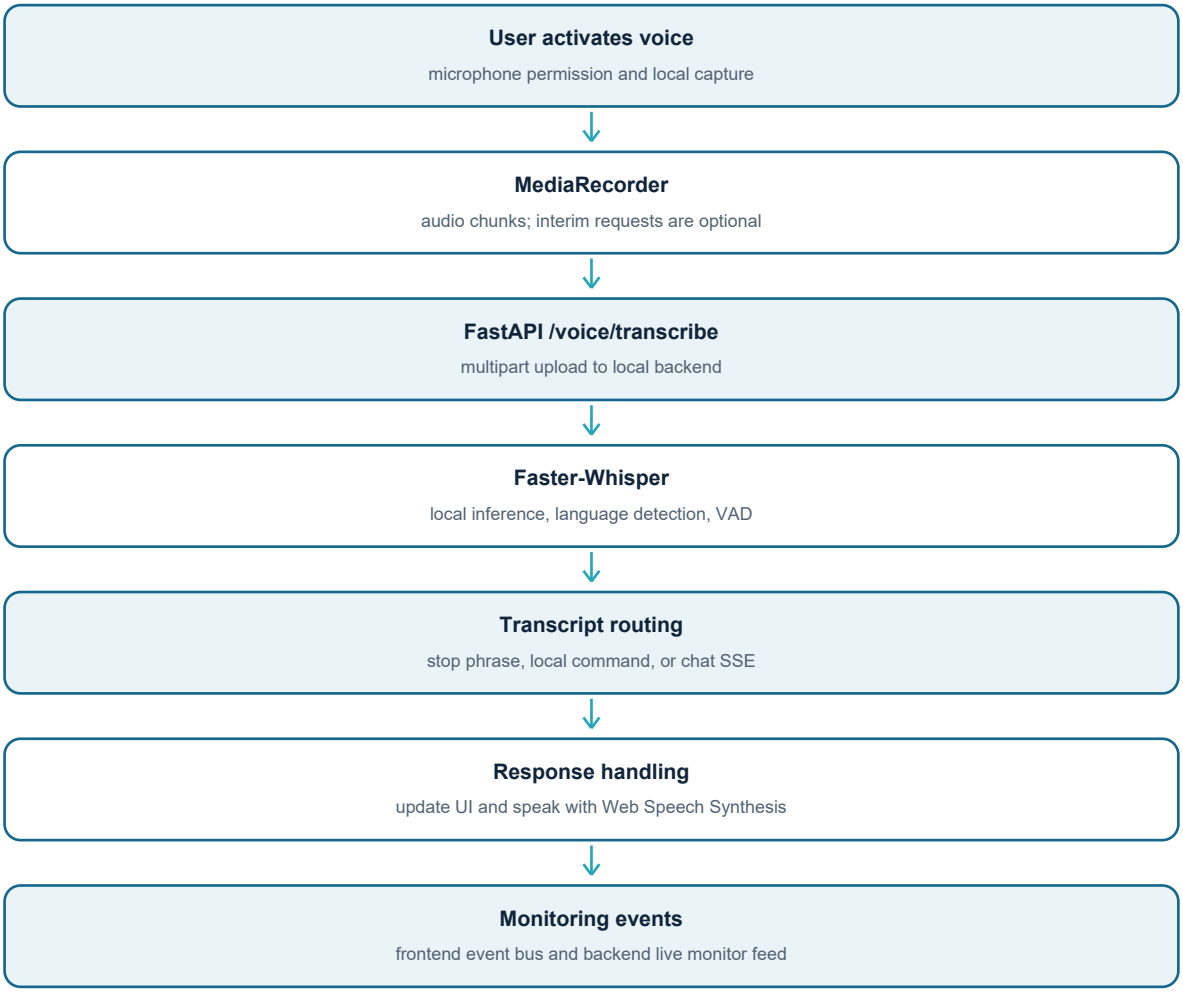


Figure 12.1. Voice command processing and response path.

13. Text command processing

Typed chat and voice transcripts share the chat transport for general questions, but explicit browser phrases are intercepted in the frontend first. parseBrowserVoiceCommand matches a finite set of English patterns: open/search/navigate, back/forward/reload, browser visibility/layout, snap positions, and pin/unpin. normalizeUrl expands a small known-site map, prefixes https for domain-like input, and otherwise constructs a Google search URL.

Matched browser commands execute against the Zustand browser store and native Tauri commands and create a local assistant message. They do not make a chat completion request. Unmatched text is sent as a POST to /api/v1/chat/completions with stream=true, a conversation ID, source metadata, and current video context. The client decodes event-stream records incrementally, appends deltas, and reacts to image_search, video_search, and browser_url metadata.

The frontend parser is deterministic and deliberately narrow. It is not a general intent model: variations outside its patterns fall through to the backend or are interpreted as ordinary chat. Errors in the chat request are surfaced to the UI; malformed SSE payloads are skipped. The browser command path has different error handling and may return a local failure message without contacting the backend.

Source references: frontend/src/voice/browserCommands.ts; frontend/src/hooks/useChat.ts; frontend/src/stores/browserStore.ts; backend/app/api/v1/routes/chat.py

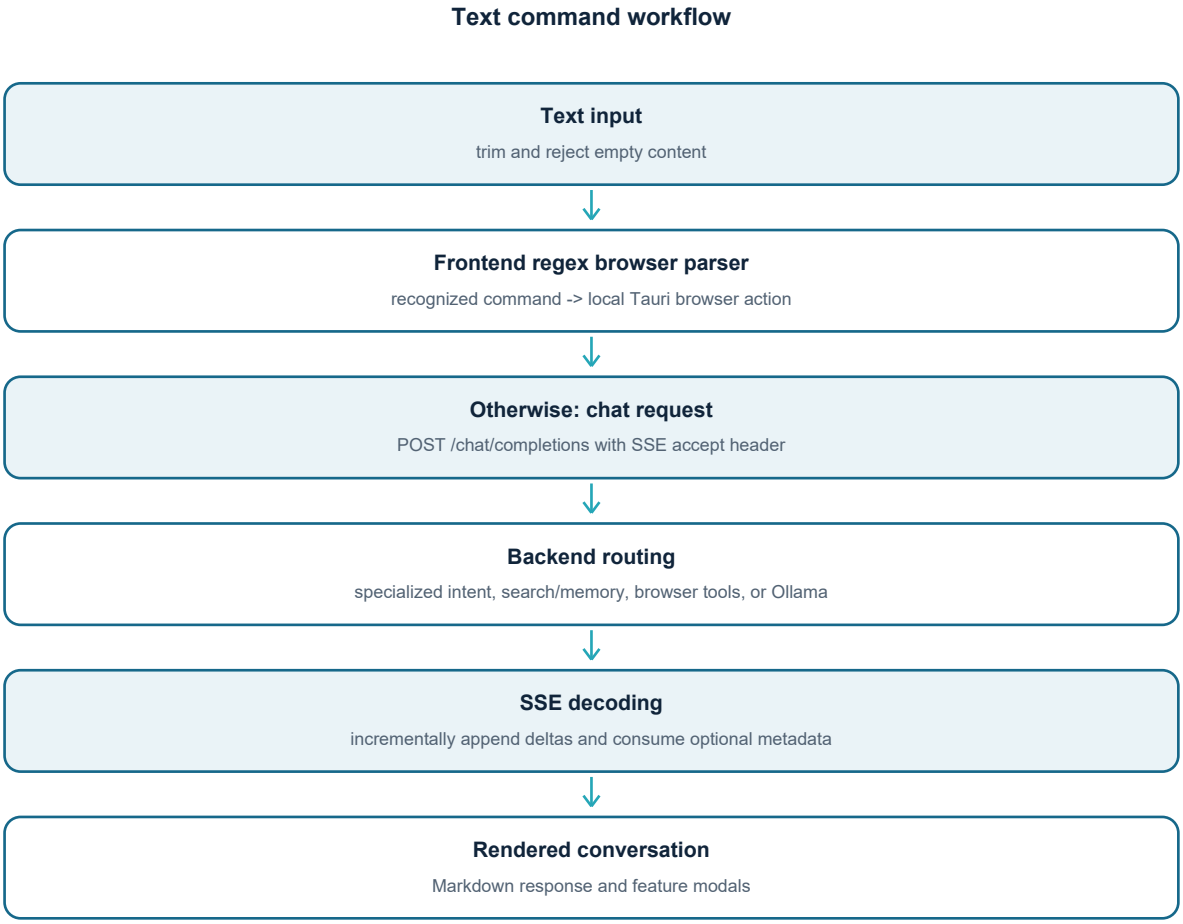


Figure 13.1. Text input bifurcates into a local browser-command path or streamed backend chat.

14. Natural-language understanding and intent recognition

Natural-language processing in this project consists of multiple modest mechanisms rather than one NLU model. The frontend parser handles a fixed vocabulary of browser commands. The backend IntentClassifier uses keyword/regex rules for greetings, identity, personal recall, browser requests, image searches, current affairs, and other intents. Specialized video and YouTube command routers perform further pattern extraction. For broad requests, the chat model interprets the prompt as free-form text.

The classifier's output determines route order and whether the system should use memory, search, browser planning, a specialized media handler, or generic generation. When current information is required, the system tries retrieval before prompting the model. Personal recall can query Chroma, while selected personal answers may be extracted and persisted after a model call and duplicate-similarity check.

This architecture supports explainable, source-inspectable rules but inherits brittleness from phrases and site DOM changes. The model's natural-language ability is not equivalent to structured intent confidence. The repository includes confidence-related components and a confidence retry path, but no training dataset or measured accuracy. Search

evidence checks are heuristic. Claims about language coverage or accuracy must therefore be framed qualitatively and qualified.

Source references: frontend/src/voice/browserCommands.ts; backend/app/chat/intent.py; backend/app/chat/routing.py; backend/app/videos/command_router.py; backend/app/youtube/youtube_command_router.py; backend/app/chat/confidence.py

15. Floating browser architecture and implementation

The floating browser has a desktop presentation layer and a separate automation layer. In React, BrowserLauncher opens/hides the panel. BrowserWindow calculates position and size, implements pointer drag and edge/corner resize, listens for tab changes, and measures the content area's bounding rectangle. It uses ResizeObserver and requestAnimationFrame to keep the native webview aligned with the React toolbar and panel.

In Rust, BrowserWindowManager stores visible/open/layout geometry and tabs behind a Mutex. browser_open validates and selects or creates a URL tab. ensure_webview attaches an external-URL child webview to the main Tauri window, updates its bounds, shows the active tab and hides other tabs. Navigation callbacks update tab URL state and emit browser-url-changed. The frontend updates its Zustand store and posts active page changes to the backend.

The Python BrowserManager connects over Chrome DevTools Protocol to the already-running embedded browser. Navigation, DOM interaction, content extraction and screenshots act on a selected page. The backend's active-page route accepts a tab_id but the central session selection is URL-based; same-URL pages can be ambiguous and the tab_id is not the authoritative selector. The backend manager does not launch an independent browser in the inspected path.

Floating layout, mini, expanded, integrated, hide/show, snapping, address/search, browser history and tab UI are represented. Some labels are more complete than native behavior: Rust snap ignores its position argument and only shows the browser; always-on-top state is recorded but not applied through a native top-most API call; close hides webviews rather than proving destruction. MiniPlayer exists but is not mounted in BrowserWindow and its postMessage protocol is not shown connected to the separate child webview.

Source references: frontend/src/components/browser/BrowserWindow.tsx; frontend/src/stores/browserStore.ts; frontend/src-tauri/src/browser/window.rs; frontend/src-tauri/src/browser/commands.rs; backend/app/browser/browser_manager.py; backend/app/browser/browser_session.py; backend/app/api/v1/routes/browser.py



Figure 15.1. Browser lifecycle from open request through native bounds synchronization and backend page selection.

16. YouTube and video automation

YouTube automation is only partially implemented. The repository has a browser-DOM command path, but it does not provide complete, reliable, or official YouTube account control.

Two related capabilities should be distinguished. Video discovery searches SearXNG's videos category, normalizes and deduplicates results from configured engines, and displays a paginated gallery. Saved video metadata is written to a dedicated Chroma collection and can be listed, searched, removed, and reopened in the embedded Aether browser. The SearXNG configuration enables YouTube and other video engines; search results still depend on upstream engine availability.

VideoCommandRouter recognizes natural-language discovery and saved-video intents, and can open/select/save results using conversation metadata. Separately, the YouTube command router and YouTubeManager use the active browser page to search, select visible results, navigate, operate playback controls, and extract information through page DOM. This is browser automation, not the YouTube Data API, OAuth account control, or an official playback SDK.

The browser-based YouTube route is partial and fragile by design: it depends on live DOM structure, visible results, browser cookies/session, and page state. Some account-related actions explicitly return unsupported responses. Ranking by view count or date is not reliably established. The UI MiniPlayer emits generic postMessage commands but is not rendered by the browser window and no bridge to the child WebView2 is present; do not claim independent media-control synchronization as verified.

Existing backend tests assert command recognition, navigation, result selection, a verified playback state, refusal of account changes, and routing order. These are test intentions only; backend tests were not run during this documentation audit.

Source references: backend/app/videos/service.py; backend/app/videos/command_router.py; backend/app/videos/storage.py; backend/app/youtube/youtube_command_router.py; backend/app/youtube/youtube_manager.py; frontend/src/components/videos/VideoSearchProvider.tsx; frontend/src/components/browser/MiniPlayer.tsx

Video discovery and open workflow



Figure 16.1. Video discovery, saved metadata, and optional in-browser playback path.

17. Other browser and desktop automation

The backend browser API supports open, active-page selection, commands, and state. Commands include navigation/search, screenshot, click/type/scroll, back/forward/reload, and state/window-control labels. The central manager attaches to a local CDP endpoint and the tool manager dispatches a fixed browser-tool list. Screenshot capture metadata and image data are stored locally; browser routes expose listing and deletion.

The frontend Files pane lists, previews, downloads, and deletes existing screenshots, but its provider does not implement a capture action. Sample prompts to say "take a screenshot" are UI instructions, not a connected Files-pane command. Capture is available through the backend browser automation command path.

The voice command executor supports a limited allowlist of applications and known folders, along with date/time responses. Arbitrary shell text is rejected; this is safer than exposing a general shell executor. Tauri's shell plugin is

configured with shell-open capability, but no broad application-launch command path is established by the inspected UI shell code.

Several older browser helpers remain in the tree for context, tabs, storage, downloads, window state, and events. They are not all wired into the central browser manager. In particular, some legacy tests expect a BrowserService facade and manager attributes that do not match the centralized implementation. The presence of these files should not be summarized as fully integrated desktop automation.

Source references: backend/app/api/v1/routes/browser.py; backend/app/browser/browser_automation.py; backend/app/browser/browser_interaction.py; backend/app/browser/window_manager.py; backend/app/voice/commands.py; frontend/src-tauri/capabilities/default.json

18. AI model integrations and inference mechanisms

OllamaProvider implements the principal language-model call. It posts a generation request to the configured Ollama HTTP API, parses newline-delimited JSON, and yields text response deltas. ResponseGenerator and PromptBuilder compose user input with conversation history, selected memory, web context, and output instructions. The chat service emits those deltas as Server-Sent Events.

The model name and endpoint are settings-driven. Repository quick-start material recommends pulling a Qwen 2.5 7B instruct model, but the source audit does not establish which model is installed or running on a machine. The documentation therefore treats Qwen as a setup recommendation, not a runtime guarantee. No external hosted-model credential or API integration is required by the shown Ollama path.

The provider manager currently contains descriptors rather than working interchangeable providers. There is no model training, fine-tuning, evaluation set, or accuracy metric in the repository. Faster-Whisper is a separate transcription model/runtime and should not be conflated with the language model. Chroma retrieval is another separate mechanism and is not an LLM.

Source references: backend/app/llm/ollama_provider.py; backend/app/llm/provider_manager.py; backend/app/chat/response_generator.py; backend/app/chat/prompt_builder.py; backend/pyproject.toml; README.md; Start.md

19. Database, storage and memory management

Relational persistence stores conversation history and browser screenshot records. ConversationHistoryStore uses SQLAlchemy models and can be configured for SQLite or PostgreSQL; local runtime data uses SQLite. Startup initializes model tables with metadata.create_all, not a versioned Alembic migration sequence. The Conversation and AuditEvent ORM models exist, but AuditLogger emits logs rather than writing durable audit rows.

Semantic memory uses Chroma upserts with content-derived IDs and metadata, supports similarity queries and recent listing, and can be populated from selected personal conversation facts after extraction and duplicate checking. Image-search history and saved-video metadata use separate persistent Chroma stores. Screenshot records include page metadata and base64 image data in SQLite. No raw database content was opened during this review; the local database files are intentionally excluded from the generated directory listing.

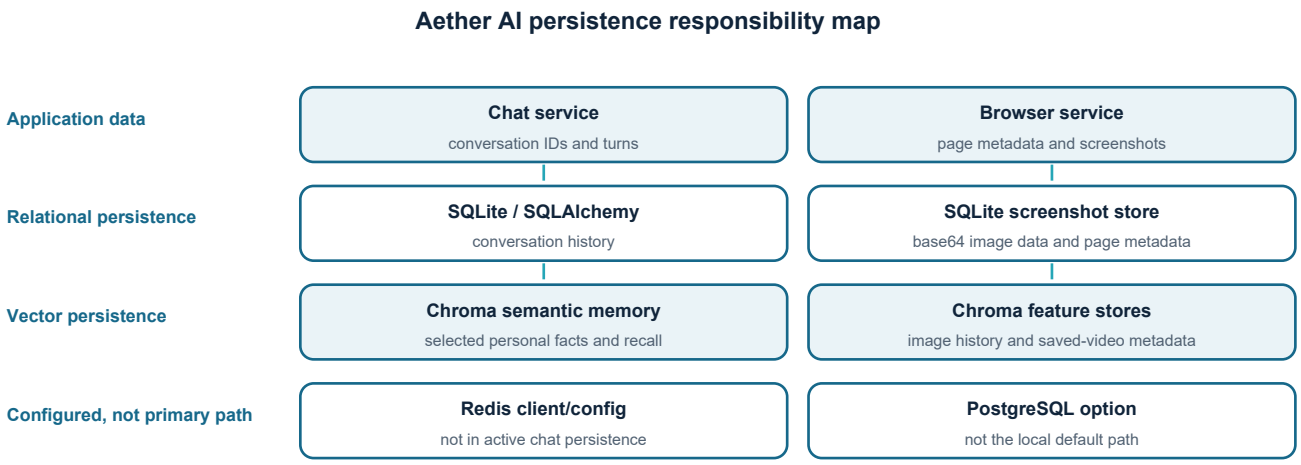
Redis is declared/configured but not on the active chat-memory path. There is no implemented user-facing API to delete all semantic memories or conversations in the audited route set, and no retention policy was identified. The project has no single relational ER model for all features: vector collections and screenshot image blobs are distinct storage shapes. The diagram below is a storage responsibility map, not an invented entity-relationship schema.

The frontend Files UI describes screenshots as in-memory or zero-disk-clutter, but backend screenshot persistence is SQLite-backed. This is an implementation/documentation mismatch with privacy and data-retention consequences.

The main avatar is integrated through AetherLogo and Avatar. Its rendered face is CSS-based; AvatarAssetManager includes asset-loading logic, but the reviewed Avatar render does not display those loaded assets. AvatarAudio opens a separate microphone stream for amplitude analysis while the voice engine may also have an active recording stream.

RegexWakeWordDetector exists but is not used by the voice engine, so no wake-word gating was verified. MiniPlayer is exported but not mounted; the mini browser layout resizes the webview instead. The frontend image modal opens results supplied in chat metadata and does not itself initiate image search.

Source references: backend/app/memory/history_store.py; backend/app/memory/chroma_manager.py; backend/app/models/conversation.py; backend/app/models/audit_event.py; backend/app/browser/screenshot_store.py; backend/app/database/session.py; frontend/src/components/files/FilesModal.tsx



Local runtime contents are intentionally not included in this document.

Figure 19.1. Persistence responsibility map; Chroma collections are not relational foreign-key tables.

Table 19.1. Persisted state and caveats

Store	Data represented	Caveat
SQLite conversation history	Conversation records and message/history payloads	Initialized via create_all; no migration execution in startup path.
SQLite screenshot store	IDs, URL/title/time and base64 image data	Persistent local files, despite frontend in-memory wording; no ongoing size cap.
Chroma semantic memory	Text facts, source/conversation metadata and embedding-backed retrieval	No project-trained embedding model or retention/deletion API established.
Chroma image history	Image-search requests/results metadata	External result URLs can expire or become unavailable.
Chroma saved videos	Video metadata and identifiers	Metadata only; video bytes are not stored.
Redis	Client/configuration helpers	Not integrated into the main chat persistence flow.
PostgreSQL	SQLAlchemy configuration option	Not the local default path; no full deployment compose in repository.

20. API documentation and integrations

HTTP routes are versioned beneath /api/v1, with a WebSocket at /api/v1/monitoring/ws. FastAPI validates request schemas and bounds selected query parameters. Chat always emits an event stream from its route; the request's stream flag does not switch the backend to a non-streaming response. API consumers should use the current route contract rather than infer behavior from request field names.

The frontend sends JSON for chat, video search, browser active-page selection, and standard resource operations. Voice transcription is multipart form data. The monitoring connection is a WebSocket delivering an initial snapshot and subsequent events. Image download returns image bytes rather than a JSON object. No API authentication mechanism is shown, so the routes must be treated as trusted local-only interfaces unless a separate deployment layer adds access control.

The following catalog lists the implemented surfaces and principal input/output shapes. It is a source-derived summary, not an OpenAPI client library. For full field validation and error semantics, consult each Pydantic schema and route implementation.

Source references: backend/app/api/v1/router.py; backend/app/api/v1/routes/*; backend/app/chat/schemas.py; frontend/src/services/apiClient.ts; frontend/src/hooks/useChat.ts

Table 20.1. Versioned API catalog; prepend /api/v1 to each HTTP/WebSocket suffix.

Method and path (suffix)	Request / response summary
GET /health	No request body; health status/service/version.
GET /system/snapshot	No body; system, model, CPU/RAM/disk, uptime, host/platform and optional GPU.

Method and path (suffix)	Request / response summary
POST /chat/completions	ChatCompletionRequest; SSE JSON chunks include delta, conversation ID, done and optional metadata.
POST /browser/open	URL and optional profile; success and current URL.
POST /browser/active-page	URL and optional tab ID; records current page context for automation.
POST /browser/command	Command plus optional target/selector/text/direction/screenshot options; command-dependent result.
GET /browser/state	No body; running/current URL/title/tab count.
POST /images/search	Query, limit 1-24, optional source/conversation; normalized results and request metadata.
GET /images/history	Optional limit 1-100; history entries.
GET /images/history/{request_id}	One stored entry or not-found response.
GET /images/download?url=...	HTTP(S) URL; returns image bytes or validation/upstream/content-type error.
POST /videos/search	Query, page 1-20 and limit 1-50; normalized results and has_more.
GET /videos/saved	Optional limit; saved metadata list.
GET /videos/saved/search	Nonblank query length 1-200 and optional limit; matching saved metadata.
POST /videos/save	VideoResult fields; persists and returns a saved item.
DELETE /videos/saved/{video_id}	Validated identifier; deleted flag or 4xx response.
GET /memory	Optional limit 1-500; memories and count.
GET /memory/search	Query length 1-500 and limit 1-100; semantic matches and count.
GET /screenshots	Optional limit 1-100; screenshot entries and count.
GET /screenshots/{id}	One screenshot, including encoded image data, or not found.
DELETE /screenshots/{id}	Delete one screenshot; success or not found.
DELETE /screenshots	Clear screenshot records; deleted count/message.
GET /voice/health	Voice engine status and non-secret model/device/compute/VAD metadata.
POST /voice/transcribe	Multipart audio file; transcript and detected language; size/model/busy errors.
POST /voice/execute?transcript=...	Transcript query parameter; handled/message/action type for local allowlisted commands.
WebSocket /monitoring/ws	Initial event snapshot, connected event and later event payloads.

20.1 Chat streaming contract

A typical browser chat body contains a messages array, stream=true, a conversation_id, and user_metadata such as source and selected video context. The response is text/event-stream. Each data line is JSON; the frontend incrementally reads the stream, parses records separated by blank lines, appends delta values, and consumes metadata for image, video, or browser actions. Malformed events are skipped by the frontend. A network or HTTP failure is rendered as an unavailable-assistant response.

```
POST /api/v1/chat/completions
Accept: text/event-stream
Content-Type: application/json

{"messages":[{"role":"user","content":"Explain the project"}],
 "stream":true,"conversation_id":"frontend-chat",
 "user_metadata":{"source":"text"}}
```

20.2 Error and event behavior

Validation failures are generally HTTP 422 through FastAPI and the registered error handler. Feature routes may return 400, 404, 429, 502, or 503 depending on input and service state. Streaming errors can be represented inside the event stream rather than as a new HTTP status after headers have been sent. The monitoring event bus is in process and not a durable event log.

21. Security and privacy mechanisms

Security controls are incomplete and should be reported explicitly. The source contains request IDs, schema validation, bounded upload sizes, command allowlists for voice actions, and some URL/content-type validation. Those controls are useful but do not constitute user authentication, authorization, rate limiting, encryption at rest, or a full threat model.

No authentication or authorization dependency is shown on the HTTP routes or monitoring WebSocket. CORS is not an access-control mechanism. Settings can bind the API broadly, although the supplied startup command binds to 127.0.0.1; deployment should explicitly retain loopback-only access or add an authenticated gateway. The Tauri config sets a null CSP. The WebView2 startup arguments include automatic acceptance of media capture prompts, which weakens microphone/camera consent and must be reconsidered before production.

The Vite development server configuration binds to 0.0.0.0, making it reachable from other interfaces on the host network unless network controls prevent access. This is a development configuration, not evidence that the production desktop build binds similarly.

The image download proxy and web retrieval path accept external URLs but do not establish destination-IP or redirect restrictions, leaving SSRF hardening incomplete. Browser interaction fallbacks construct page JavaScript from selector/text inputs. The local CDP debugging port is privileged automation access and should remain local and unavailable to untrusted processes. Browser page contents and transcripts may contain sensitive information.

EncryptionService is a placeholder, PermissionManager currently denies all permission checks, and AuditLogger does not persist to the AuditEvent table. Logging may include transcripts or extracted personal facts. The local Docker/SearXNG setup uses a fixed development secret; this document deliberately does not reproduce it. Environment files and local database contents were not read or included. Replace development secrets, protect local data files, and define retention before any sharing.

Voice command code rejects arbitrary shell execution and restricts supported commands, but the HTTP endpoint itself has no authentication. User consent and deployment boundaries are therefore still essential. Features that fetch or automate web pages should run with least privilege and explicit interaction confirmation where an action has consequences.

Source references: backend/app/security/*; backend/app/middleware/*; backend/app/api/v1/routes/images.py; backend/app/web/search.py; backend/app/browser/browser_interaction.py; frontend/src-tauri/src/main.rs; frontend/src-tauri/tauri.conf.json; docker/searxng/*

Table 21.1. Privacy/security assessment

Area	Observed protection	Gap / operational guidance
API access	Pydantic validation and CORS configuration	No auth/rate limits; CORS is not authorization. Keep API loopback-only.
Voice action execution	Allowlisted applications/folders; arbitrary shell rejected	No auth on route; do not expose to untrusted clients.
Secrets	Settings abstraction and env-file patterns	Encryption/secret manager is not implemented; local SearXNG dev secret must be replaced.
Browser	URL parsing, local CDP, Tauri permission manifest	Debug endpoint is powerful; CSP is null; media prompt bypass is configured; child navigation has no allowlist.
External links	Tauri shell-open integration for Markdown links	URL normalizer preserves arbitrary schemes; restrict protocols before opening external targets.
External fetch	HTTP(S) and content-type checks for image download	No comprehensive IP/redirect/size safeguards; harden SSRF and response limits.
Data retention	Local SQLite/Chroma persistence	No identified retention controls for conversation/memory; protect and purge local stores.
Audit	Structured logs and in-process monitoring events	Audit model is not written durably; logs may contain personal/transcript data.

22. Feature-by-feature implementation details

This chapter consolidates feature maturity so a report writer can avoid turning an interface into a claim of completed functionality. The status column reflects code wiring and the boundary described in the cited modules; a row labelled implemented can still depend on local services or external sites.

Source references: Primary feature modules in backend/app and frontend/src.

Table 22.1. Feature implementation matrix

Feature	Status	Inputs and internal flow	Limitations
Chat completion	Implemented	User messages -> orchestrator -> search/memory/specialist or Ollama -> SSE.	Every request is streamed; request feature flags are not all honored.
Conversation history	Implemented	Conversation ID -> SQLAlchemy history store -> prompt context.	No user auth or retention controls identified.
Semantic memory	Implemented	Selected personal facts -> Chroma upsert -> similarity recall.	Selective heuristics/model extraction; no memory deletion route.
Voice transcription	Implemented	MediaRecorder -> multipart local API -> Faster-Whisper -> transcript.	Needs microphone, local backend/model; no wake-word gate in engine.
Voice command	Partial	Transcript -> local executor -> allowlisted app/date/folder action.	Limited commands, unauthenticated route, no arbitrary shell.
Browser command parsing	Implemented, narrow	Frontend regex -> Zustand -> Tauri invoke.	Fixed command grammar; nonmatches route to chat.
Tauri browser	Implemented	Rust manager -> child WebView2 -> bounds/tab events.	Windows-centric; topmost/snap command semantics incomplete.
Playwright automation	Partial	Python manager attaches via CDP to active page.	Assumes CDP target; duplicate URL selection ambiguity.
Web retrieval	Implemented, heuristic	SearXNG results -> extraction/ranking/cache/citations -> model context.	Evidence validation not robust; SSRF safeguards incomplete.
Image search/history	Implemented	SearXNG image category -> normalize/rank -> Chroma -> modal.	External result availability; download destination controls incomplete.
Video discovery	Implemented	SearXNG video category -> dedupe/normalize -> UI and Chroma save.	Search engine dependent; has_more/ranking are limited.
YouTube control	Partial	Command router -> active page DOM -> navigate/select/play/info.	DOM/cookie dependent; account operations unsupported.
Screenshot capture/library	Implemented	CDP screenshot -> SQLite image data -> REST list/delete -> UI.	Persistent storage; frontend copy suggests memory-only.
Live monitoring	Implemented, in process	Backend event bus/WebSocket and frontend local voice event bus.	Not durable telemetry; frontend does not show reconnect logic.
System telemetry	Partial	psutil host metrics -> snapshot -> React Query poll.	Some status fields are fixed labels; GPU may be unavailable.
Plugin execution	Planned/interface	Plugin showcase contains technology cards.	No executable backend plugin registry/runtime.
RAG engine	Planned/interface	Directory/class scaffold exists.	No implemented end-to-end RAG ingestion/query engine.
Workflow engine	Planned/interface	Initialization scaffold exists.	No graph/task execution pipeline.
Dashboard/modules	Placeholder	Root route and generic module route exist.	Dashboard is null; nested Outlet is not rendered by layout.
Metrics/workspace/dock/keyboard hooks	Placeholder	Hooks are exported.	Return empty objects and do not implement the advertised services.

23. Important classes, functions and source references

The following map is intended to make source review and later report citation straightforward. Paths are relative to the repository root. Function/class names identify the implementation entry point; module boundaries are more reliable than README descriptions where the two differ.

Source references: Source paths were checked against the repository inventory and implementation review.

Table 23.1. Key implementation references

Concern	File and symbol(s)	Why it matters
Application lifecycle	backend/app/main.py: lifespan(), create_app()	Logging, container, DB initialization, voice startup, middleware/router wiring.
Dependency graph	backend/app/core/container.py: ApplicationContainer	Constructs chat, browser, memory and service dependencies.
API router	backend/app/api/v1/router.py; backend/app/api/v1/routes/*	Concrete HTTP/WebSocket route contract.

Concern	File and symbol(s)	Why it matters
Chat contract	backend/app/chat/schemas.py: ChatCompletionRequest; routes/chat.py	Request validation and SSE response boundary.
Intent and route	backend/app/chat/intent.py: IntentClassifier; routing.py: RoutingEngine	Rule-based intent recognition and specialized routing.
Chat orchestration	backend/app/chat/orchestrator.py: ConversationOrchestrator.stream_response	Specialized command ordering, context, model and retrieval paths.
Model generation	backend/app/llm/ollama_provider.py: OllamaProvider; chat/response_generator.py	Local Ollama request and generated-response context.
Planning and tools	backend/app/agents/planner.py: PlaceholderPlanner; tools/manager.py: PlaceholderToolManager	Narrow browser planning/dispatch; no general agent tool ecosystem.
Browser automation	backend/app/browser/browser_manager.py: BrowserManager; browser_session.py	CDP attach and page/session selection.
Search	backend/app/web/search.py	SearXNG search, retrieval, ranking, citations and cache.
Voice backend	backend/app/voice/service.py: VoiceService; voice/commands.py: VoiceCommandExecutor	Faster-Whisper inference and allowlisted local commands.
Video / YouTube	backend/app/videos/service.py: SearXNGVideoSearchService; backend/app/youtube/youtube_manager.py: YouTubeManager	Search/persistence and separate page-DOM control.
Persistence	backend/app/memory/history_store.py: ConversationHistoryStore; chroma_manager.py	Conversation history and semantic memory operations.
Monitoring/security	backend/app/monitoring/service.py: MonitoringService; app/security/*	In-process event bus; security interfaces include placeholders.
Frontend composition	frontend/src/App.tsx: App; routes.tsx: router; DesktopLayout.tsx	Provider hierarchy, route map and visible desktop shell.
Chat client	frontend/src/hooks/useChat.ts: useChat; ChatPanelAether.tsx	Browser command interception, SSE consumption and chat UI.
Voice client	frontend/src/voice/engine.ts: AetherVoiceEngine; providers.ts	Frontend recording state machine, local transcription and browser TTS.
Browser command parser	frontend/src/voice/browserCommands.ts: normalizeUrl(), parseBrowserVoiceCommand()	Deterministic local browser grammar.
Native browser	frontend/src-tauri/src/browser/window.rs: BrowserWindowManager, ensure_webview(); commands.rs	Native child webview lifecycle and invoke commands.
Video UI	frontend/src/components/videos/VideoSearchProvider.tsx; VideosModal.tsx	Search, state, filtering, saving and opening selected results.

24. Internal workflows and architecture diagrams

The earlier chapters include vector diagrams for the logical architecture, chat path, voice path, text routing, Tauri browser lifecycle, and video search. Those diagrams are deliberately simple: they show application-owned boundaries and avoid suggesting unverified distributed components, databases, or machine-learning stages.

A request can have a different route depending on where recognition happens. Explicit browser text is matched by a frontend regex and runs locally through Tauri. A video phrase may be recognized in backend specialized routing and trigger SearXNG. General text reaches the backend chat orchestrator. Spoken input is first transcribed, then either executed as a narrow local command or forwarded to the same chat stream. These are related but not identical pipelines.

Monitoring is an observability side path. Voice events are published into a frontend event bus and displayed locally; backend events use an in-process service broadcast over WebSocket. A frontend monitor connection can fail independently of chat. The backend ring buffer is transient, so it should not be represented as durable audit storage.

Source references: frontend/src/hooks/useChat.ts; frontend/src/voice/engine.ts; backend/app/chat/orchestrator.py; backend/app/monitoring/service.py; frontend/src/hooks/useMonitoring.ts

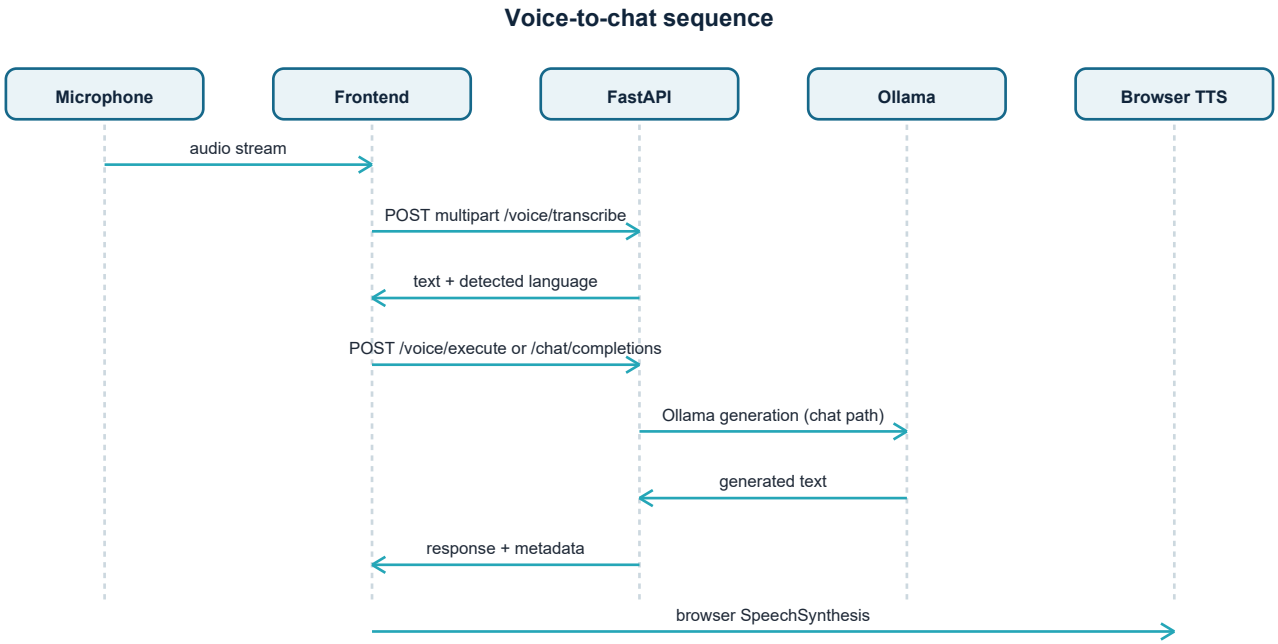


Figure 24.1. Cross-process voice/chat sequence.

25. Testing methodology and actual test cases

The repository contains pytest backend tests and Vitest/Testing Library frontend tests. The backend test files cover health, chat, orchestration/routing, web search, memory, persistence, monitoring, browser, screenshots, videos, voice, and YouTube. The tests use fakes/mocks for external services in many cases and assert behavior such as input validation, stream records, routing, persistence operations, command parsing, and error responses. Test presence describes intended contracts, not actual pass status.

For this documentation task, the frontend test suite was run from the frontend directory on 13 November 2026 with `npm test -- --reporter=dot`. Vitest reported 13 test files: 12 passed and one failed; 54 tests passed and one failed. The failure is `VoiceIntegration.test.tsx` rendering ChatPanelAether without the required VideoSearchProvider; `useVideoSearch` throws its context error. This is the observed result, not a claim that the application runtime fails when App mounts its providers. Warnings/logs appeared during tests, including a Vite config-loader warning and expected asset fallback logging.

Backend tests were not executed as part of this source-only documentation audit. Consequently no backend test pass count is asserted. Static review found that `test_browser.py` contains expectations for a legacy BrowserService shape that does not match the centralized manager. Several route surfaces also lack direct route-level tests in the listed test set. CI status and production build status were not independently verified in this run.

The `TopNavAether.test.tsx` file renders a literal `<div>Aether</div>` and does not exercise a navigation component. Other tests are focused on components and deterministic helpers; no full route/browser integration tests are present in the frontend suite.

Source references: backend/tests/*; frontend/src/**/*.test.*; frontend/package.json; frontend/vitest.config.ts

Table 25.1. Test areas and observed verification

Suite / file	Test intent	Audit result
frontend voice/engine.test.ts	Local recording, backend transcript request, chat handoff, permission denial, cancellation.	Included in run; overall run has 54 passed, 1 failed.
frontend voice/browserCommands.test.ts	URL/site normalization; open/search/navigation/window/snap parsing.	Included in run.
frontend voice/formatter.test.ts; languages.test.ts	Markdown-to-speech cleanup and language/voice selection.	Included in run.
frontend voice/monitoring.test.ts; VoiceIntegration.test.tsx	Event de-duplication, voice state monitoring, microphone UI.	One VoiceIntegration test fails because test does not mount VideoSearchProvider.
frontend avatar tests	Asset fallback, emotion blending and lip-sync inputs.	Included in run.

Suite / file	Test intent	Audit result
frontend TopNavAether.test.tsx	Navigation UI	Does not render a navigation component; asserts only a literal div.
frontend Markdown/scroll tests	Custom links, markdown blocks, scroll behavior.	Included in run.
backend test_health.py	Health HTTP status and `ok` status body.	Source inspected; backend suite not run.
backend test_chat.py / test_orchestration_routing.py	SSE, prompt context, model options, routing, retrieval fallback, memory extraction.	Source inspected; backend suite not run.
backend test_web_search.py	Trigger rules, citation formatting, caching, snippet fallback and ranking.	Source inspected; backend suite not run.
backend test_memory.py / test_persistence.py	Semantic memory operations and SQLite conversation round-trip.	Source inspected; backend suite not run.
backend test_monitoring.py	Event delivery, service wiring, WebSocket initial messages.	Source inspected; backend suite not run.
backend test_videos.py / test_youtube.py	SearXNG result normalization, saved data, command routing, browser/YouTube actions.	Source inspected; backend suite not run.
backend test_voice.py	Date/time/app allowlist, dangerous command rejection, model settings and API cases.	Source inspected; backend suite not run.
backend browser/screenshot tests	CDP page selection, screenshot capture/storage, browser state, legacy facade.	Source inspected; legacy test compatibility concern noted.

26. Setup, configuration and operating model

The repository quick-start expects Python 3.13+, Node.js 22+, npm 10+, and Rust/Cargo for desktop builds. Install the backend package with its development extra and frontend dependencies from `frontend/`. The backend is started from the backend directory with a factory-based Uvicorn command; the supplied command binds to loopback on port 8001. The frontend dev server uses port 5173. Tauri development invokes the frontend dev command before launching the desktop shell.

Ollama must be installed and running for the local LLM path. Start.md recommends a Qwen 2.5 7B instruct model, but the active model is controlled by backend configuration. SearXNG is optional for general/image/video retrieval and is available through the repository's Docker Compose file on port 8080. The backend expects a local voice model; first startup may require model download before health reports ready.

Useful frontend configuration includes VITE_AETHER_API_URL and VITE_AETHER_WS_URL. Backend settings cover database, Chroma, Redis, SearXNG, Ollama, browser, and voice options. Voice settings documented in backend/README.md include model size, device, compute type, and CPU thread count. Environment files were not read; do not copy local secret values into reports.

Source references: README.md; Start.md; backend/README.md; frontend/package.json; backend/pyproject.toml

```
# Backend (from backend/)
uvicorn app.main:create_app --factory --host 127.0.0.1 --port 8001

# Frontend (from frontend/)
npm run dev

# Optional Tauri desktop development (from frontend/)
npm run tauri:dev
```

The command snippets explain documented local development workflow only. They are not evidence that the services were launched or a production bundle was generated during this review.

27. Technical challenges and engineering solutions

Maintaining alignment between a React panel and a native child WebView2 surface is a desktop-specific challenge: the child surface is not part of the DOM and cannot follow ordinary CSS transforms. The implementation responds by measuring the committed content box, observing size changes, synchronizing bounds, and hiding the child surface while dragging. These mechanisms address layout timing and pointer interception, though native behavior still requires Windows integration testing.

Streaming responses require the client to handle partial transport chunks that may not align with SSE event boundaries. useChat keeps a text buffer, splits complete blank-line-delimited events, parses JSON data, and accumulates deltas. Voice state also spans asynchronous microphone setup, server readiness, model loading, transcription, chat streaming, speech playback, and optional relistening; AetherVoiceEngine uses explicit phases, timers, generation IDs, and cleanup.

Local inference can take time to initialize and needs visible readiness/error states. The voice client polls the health route and waits for model loading before recording. SearXNG/website availability and browser DOM changes are external dependencies; the code normalizes results and provides fallbacks, but cannot guarantee ranking or stable selectors. Testing uses mocked integrations for deterministic unit cases; deployment-specific end-to-end verification remains necessary.

The codebase also reveals coordination costs from parallel state stores and legacy modules. Rust native browser state, frontend Zustand state, and backend CDP page selection each maintain different facts. URL synchronization helps but does not make them identical. More explicit ownership, stable tab IDs across layers, typed event contracts, and integration tests would reduce state drift.

Source references: frontend/src/components/browser/BrowserWindow.tsx; frontend/src-tauri/src/browser/commands.rs; frontend/src/voice/providers.ts; frontend/src/voice/engine.ts; frontend/src/hooks/useChat.ts; backend/app/browser/browser_session.py

28. Results, current status and known limitations

The repository has a usable foundation for local chat, voice transcription, SearXNG retrieval, persistent conversation/memory/media metadata, system telemetry, monitoring events, and an embedded browser. The strongest evidence is concrete API/service code, frontend flows, and focused unit tests. The frontend suite run during this documentation task yielded 54 passing tests and one failing test as described in Chapter 25.

The code does not establish a completed enterprise-grade autonomous agent. General workflow and RAG modules are stubs; plugin execution is absent; model-provider switching is not implemented; permission management and encryption are placeholders; and audit persistence is incomplete. Search validation is heuristic. Some API options in the chat request schema are not actually honored by the orchestrator/service. The system snapshot includes static status labels.

Browser features have two control planes and partial state fidelity. Python automation attaches to a local CDP target instead of launching a standalone browser. Backend active-page choice is URL-based rather than a stable cross-layer tab ID. Some browser window commands update state rather than applying a native window operation. YouTube DOM automation is dependent on a changing third-party website. MiniPlayer is not integrated into the displayed child webview.

Operational gaps include no API authentication or rate limiting, broad default bind configuration unless launch options override it, null Tauri CSP, a WebView2 media permission bypass, SSRF hardening gaps in external fetches, no identified data-retention policy, and local logs that may contain user material. Direct route tests are absent for several API areas, and backend tests were not executed during this audit. No performance or accuracy claims are made.

Repository documentation overstates some completeness: the root README describes an architecture and docs layout that do not exactly match the tracked snapshot, and frontend components contain inert controls. The generated reference deliberately distinguishes source behavior from these labels.

Source references: Repository-wide source audit; frontend test run; backend test inventory.

29. Future enhancements and scope

A safe next phase would establish an explicit threat model and production deployment boundary before expanding automation. Add authentication, rate limiting, loopback defaults, URL destination/redirect controls, response-size limits, a restrictive Tauri CSP, and explicit microphone consent. Redact transcripts and personal facts from routine logs and define retention/deletion controls for SQLite and Chroma.

Improve architecture integrity by introducing versioned database migrations, disposing async engines on shutdown, caching dependencies where appropriate, and adding contract tests around API and browser-state synchronization. Give each browser tab a stable identifier shared across React, Rust, and Playwright instead of relying on URLs. Clarify close/minimize/topmost semantics and connect or remove the unused MiniPlayer protocol.

For intelligence, implement a real permission-aware tool registry, confirmation for consequential actions, bounded task execution, and failure recovery before describing the product as autonomous. Replace evidence-presence checks with source relevance/conflict evaluation and include citations as structured response metadata. Add observable model selection, reproducible evaluation sets, and documented language coverage without claiming unmeasured accuracy.

Complete or remove placeholder dashboard, module, metrics, workspace, dock, keyboard, RAG, workflow, plugin, and security interfaces. Repair the failing frontend test by mounting required providers in the test fixture, then add focused route-level tests and a safe, isolated backend test database strategy. This is a proposed roadmap, not a committed or implemented feature set.

Source references: Recommendations based on gaps in source; not current capabilities.

30. Technical glossary

The terms below are used in their specific Aether AI implementation context. A concept appearing in this glossary does not imply that every subsystem bearing the term is fully implemented.

Table 30.1. Glossary

Term	Definition
API	Application Programming Interface; the HTTP and WebSocket boundaries used by the frontend and backend.
Async	A programming style that lets I/O operations yield while waiting, used by FastAPI and Python services.
CDP	Chrome DevTools Protocol; browser debugging/control interface used by Playwright to attach to an existing page.
ChromaDB	Vector database used here for semantic memories and selected image/video metadata.
CORS	Cross-Origin Resource Sharing; browser policy configuration, not user authentication.
DOM	Document Object Model; the structured page representation used by browser automation.
Embedding	Numeric vector representation used for similarity retrieval; the repository does not train a project embedding model.
FastAPI	Python web framework used to construct the backend API.
Faster-Whisper	Optimized Whisper speech-to-text implementation used by the local transcription service.
LLM	Large Language Model; the configured Ollama model generates text responses.
Local inference	Model computation on a local machine/service; it does not itself provide encryption or access control.
MediaRecorder	Browser API that records microphone audio into chunks for upload.
NLU	Natural-Language Understanding; in Aether, mainly deterministic rules plus generative interpretation, not a trained intent classifier.
Ollama	Local model runtime contacted by the backend over HTTP.
ORM	Object-Relational Mapper; SQLAlchemy maps Python model classes to relational tables.
Playwright	Browser automation library; the backend uses its connection to the selected CDP browser.
RAG	Retrieval-Augmented Generation; a general pattern combining retrieval context with model generation. The dedicated Aether RAG module is a placeholder.
SearXNG	Self-hostable metasearch service used for web, image, and video categories.
SSE	Server-Sent Events; one-way streaming HTTP response used for chat deltas.
SQLite	Embedded relational database used for local conversation and screenshot persistence.
Tauri	Desktop application framework that hosts a web frontend and exposes native commands through Rust.
VAD	Voice Activity Detection; used by Faster-Whisper configuration to filter speech regions.
WebSocket	Persistent two-way connection used by the monitoring endpoint.
WebView2	Microsoft Edge-based embedded webview runtime used by the native desktop browser surface.
SSRF	Server-Side Request Forgery risk class; relevant when a backend fetches a user-supplied URL without destination restrictions.
State machine	Explicit set of states and transitions; used by the frontend voice engine to manage the interaction lifecycle.
Vector store	Storage and similarity-query layer for embedding-associated records; distinct from SQL tables.

Appendix A. Source verification notes

The application source was not modified for this deliverable. The report uses code, configuration, test names/assertions, and repository documentation as evidence. README statements about current capabilities were cross-checked against source. The backend audit inspected backend application modules and tests without opening .env or persisted database contents. The frontend review covered interface source, tests, Tauri source/configuration, and the repository's startup instructions.

All references use repository-relative paths. Tests are described as assertions unless a result is explicitly stated from the frontend test run. No model accuracy, performance, availability, training-data, or production-readiness statistic has been inferred. No credential value, API key, token, password, or local database record is reproduced.

Table A.1. Evidence classification used in this document

Label	Interpretation
Implemented	A concrete source path is wired into an API/UI/service; deployment dependencies may still be required.
Partially implemented	Source exists but coverage, integration, policy, environment support, or behavior is bounded.
Planned/interface	A scaffold, interface, description, or no-op is present without an end-to-end execution path.
Unsupported	The code explicitly rejects the action or no implementation was identified in the reviewed scope.
Test assertion	A property encoded in test source; not a passing runtime result unless the document states the suite was run.

Appendix B. API route source index

Table B.1. Route modules for direct schema and error details

Feature	Route source
Health/system	backend/app/api/v1/routes/health.py; backend/app/api/v1/routes/system.py
Chat	backend/app/api/v1/routes/chat.py; backend/app/chat/schemas.py
Browser	backend/app/api/v1/routes/browser.py; backend/app/browser/schemas.py
Images	backend/app/api/v1/routes/images.py; backend/app/images/schemas.py
Memory/screenshots	backend/app/api/v1/routes/memory.py; backend/app/api/v1/routes/screenshots.py
Monitoring	backend/app/api/v1/routes/monitoring.py; backend/app/monitoring/service.py
Videos	backend/app/api/v1/routes/videos.py; backend/app/videos/schemas.py
Voice	backend/app/voice/routes.py; backend/app/voice/schemas.py
Router and lifecycle	backend/app/api/v1/router.py; backend/app/main.py

Appendix C. Test inventory and limits

Backend test files present: test_browser.py, test_browser_active_page.py, test_browser_window_system.py, test_centralized_browser.py, test_chat.py, test_health.py, test_memory.py, test_monitoring.py, test_orchestration_routing.py, test_persistence.py, test_screenshot_store.py, test_videos.py, test_voice.py, test_web_search.py, and test_youtube.py. These files establish planned test coverage areas; the suite was not run for this documentation task.

Frontend test files present: TopNavAether.test.tsx; AvatarAssetManager.test.ts; AvatarLipSync.test.ts; EmotionEngine.test.ts; CustomLink.test.tsx; MarkdownMessage.test.tsx; VoiceIntegration.test.tsx; useSmartScroll.test.ts; browserCommands.test.ts; engine.test.ts; formatter.test.ts; languages.test.ts; and monitoring.test.ts. The run result and the single context-provider failure are recorded in Chapter 25.

Run backend tests only against an isolated test configuration/database; this repository snapshot contains pre-existing modified local SQLite files. Do not treat a test run against user data as a safe substitute for a disposable database fixture.

Appendix D. Reading the diagrams

All diagrams are generated as vector drawings inside the PDF. Labels remain selectable/searchable text. Arrows indicate the principal direction of control or data flow; they do not imply that all components execute in the same process. The browser diagram intentionally separates the native Tauri child webview from backend Playwright/CDP control. The storage diagram is an architectural responsibility map and not an ER diagram because the project also uses vector collections and image blobs outside relational entity relationships.